The Relay — Engineering Report

Vaani ML Engineering Intern take-home | Evidence cut: 13 November 2026

Evidence labels: [REAL] Groq; [SIM] stand-in simulation; [MOCK] deterministic extractive provider; [RETRIEVAL] BM25 with no model call; [DOC] provider documentation. results.csv contains 1,085 request-level rows. Blank fields mean not measured.

1. What I built and why

The Relay answers questions using exactly three papers: *Attention Is All You Need* (arXiv:1706.03762 v7), *Moshi* (arXiv:2410.00037 v1), and *Dynamic Chunking* (arXiv:2507.07955 v2). The pipeline validates/classifies, retrieves through `search_papers`, checks evidence sufficiency, and branches to one query-expansion loop or a zero-call refusal. Supported questions go through cited generation, deterministic citation checks, an LLM verifier, and at most one correction loop. A gateway adds bounded priority queuing, token-bucket admission, reservation/settlement, shared cooldown, and bounded jittered retries. The main trade-off is early capacity shedding rather than unbounded queues.

2. The real service and my stand-in

[REAL] Five Groq baseline requests exposed an 8,000 TPM and 1,000 requests/day header. Headers imply ~133.3 tokens/s refill and 86.4 s/request (inferred, not observed after a 429). **[DOC]** RPM remains unresolved: documentation/model-page values conflict (30 vs 1,000), with no RPM response header. The 200,000 daily-token limit is documented, not observed in headers. Real in-flight capacity and real 429 recovery were not measured. The stand-in simulates reasoning/content streaming, load slowdown, 429s and provider faults; concurrency and rejected-attempt accounting are assumptions, not claims about Groq.

3. Results

[REAL baseline, n=5] Content first-token p50 was 2359 ms; total p50 was 2359 ms. **[REAL quality, Setting A only]** Latest run `qa-A-2` graded 21 questions: 1 correct, 1 wrong, 16 answerable questions wrongly refused, and 3 unsupported questions correctly refused. It recorded 59,902 input+output tokens and 2 retries. Earlier `qa-A-1` wrongly refused 18 and correctly refused 3. No real B/C quality run exists. **[MOCK]** A/B/C extractive tests validate plumbing only, not Groq answer quality.

Load scenario [SIM]	Requests	OK	Rate-limited	Timeout/error
steady	28	28	0	0
burst	32	17	15	0
overload	60	29	31	0
limit_halved	21	17	4	0
daily_exhausted	30	12	18	0

Fault injection [SIM, latest run]: 5xx 21/21; timeout 21/21; incomplete stream 21/21; mixed 20/21 completed. Steady load completed all 28 requests; burst, overload, halved-limit and daily-exhaustion runs shed requests as designed.

4. Where the time goes

The slowest burst request (`lt-main-burst-0015`, **[SIM]**) took 24,479 ms; first token arrived at 24,271 ms. Queue wait was 12,016 ms, limiter pacing 10,320 ms, model wait before first token 1,926 ms, and generation after first token 208 ms. Most latency was waiting before generation, not token output. This is a stand-in trace, not a real Groq request.

5. When a limit is hit

The gateway reserves estimated tokens before sending and settles actual usage afterward. A 429 `Retry-After` creates a shared cooldown. If the required wait exceeds the deadline, the request is rejected as `rate_limited`; deadline expiry maps to `timeout`. Daily exhaustion fails fast. The user receives a capacity message and can retry later; no weaker model is silently substituted. Real Groq 429 recovery remains unmeasured.

6. Retrieval: quality versus cost

[RETRIEVAL] `rb-2` evaluates 21 questions (18 answerable, 3 unsupported) without an LLM. A uses 300-token chunks/top-3; B uses 600/top-5; C adds lexical re-ranking to B.

Setting	Hit@k	Full evidence	MRR	Mean context tokens
A	16/18	14/18	0.718	1,031
B	15/18	14/18	0.722	2,147
C	16/18	15/18	0.796	1,850

C has the strongest MRR/full-evidence recall; A uses fewer context tokens. Differences are small, and real answer quality was measured only for A, so no overall winner is established. The sufficiency gate blocked 3/3 unsupported questions but also 2/18 answerable questions.

7. How this library will mislead a naive retriever

Measured failure cases: shared vocabulary can rank another paper's optimizer passage above the Transformer passage; comparison queries can retrieve passages from only one paper; and the Transformer paper reports English-to-French BLEU as 41.0 in prose versus 41.8 in a table on the same page. A citation alone does not resolve that conflict. “Dynamic chunking” in the paper means learned segmentation, not retrieval chunk size.

8. Retries

The policy uses jittered exponential backoff, honours `Retry-After`, caps attempts, avoids retrying invalid/daily-exhausted requests, and releases concurrency slots while sleeping. **[SIM]** Retry2: hardened completed 51/120; immediate retry completed 23/120; the no-count-rejected-attempts control completed 55/120. Whether Groq counts rejected attempts against RPM is unknown, so this comparison is conditional.

9. What the person sees

Unsupported questions receive an explanatory refusal when evidence is insufficient. Capacity exhaustion returns `rate_limited` with a later-retry message. The service does not invent a reset time or silently switch models.

10. What I'm unsure about, what I left out, what I learned

Unresolved: true RPM (documentation conflict), rejected-attempt accounting, in-flight capacity, and real 429 recovery. The re-ranker is lexical, not a neural cross-encoder. The clearest engineering lesson in these runs is that retrieval recall and answer quality are different: the real A run still wrongly refused 16/18 answerable questions. The clearest engineering lesson in these runs is that retrieval recall and answer quality are different: the real A run still wrongly refused 16/18 answerable questions.

11. Feedback on Vaani

What stood out to me

• Vaani is working on an interesting problem: making voice AI practical and reliable for real-world business applications, and how the platform applies voice AI to customer conversations, lead qualification, outbound sales, inbound enquiries, and education admissions.

• The focus on multilingual conversations, low-latency responses, and real-world telephony makes the product particularly interesting.

What was unclear or could be improved

While exploring the website, I noticed several UI/UX and visual-design inconsistencies:

• **Header overlapping headings:** The navigation bar overlaps with section headings, making some content difficult to read.

• **Product Hunt badge overlapping CTAs:** The promotional badge overlaps the primary call-to-action buttons, creating visual clutter.

• **Heading clipping:** Some large headings appear partially hidden or clipped behind the navigation.

• **Hero vertical spacing:** The spacing between navigation, headings, video, and CTA buttons feels inconsistent and cramped.

• **Customer-story hierarchy:** Multiple competing metrics make it difficult to identify the most important information.

• **Section spacing consistency:** Vertical spacing and transitions between sections feel inconsistent.

• **Responsive positioning:** Fixed and floating elements should be tested across different screen sizes to prevent content overlap.

What I would improve

• Improve the design, because these are things users should never have to notice. If users are noticing layout problems like overlapping elements, it indicates poor design execution.

• I believe Vaani would benefit from hiring me or someone with a good eye for design who can identify these issues and fix them, ensuring a more polished and consistent user experience.

12. How I worked with the assistant

I used an AI assistant as a development and debugging partner throughout the project. I did not treat its suggestions as automatically correct; I reviewed the proposed implementations, ran tests, examined experiment results, and made decisions based on the observed behaviour.

Three prompts that were particularly useful:

1. I asked the assistant to help design a multi-step research question-answering pipeline with retrieval, evidence validation, citation verification, and refusal handling.

2. I asked it to investigate how rate limits, token budgets, retries, and request queuing should be handled when working with a hosted LLM provider.

3. I asked it to analyse evaluation results and investigate why the system was incorrectly refusing answerable questions.

An issue I identified: The evaluation results showed that the system was incorrectly refusing many answerable questions. This demonstrated that successful retrieval did not necessarily mean the complete question-answering pipeline was working correctly. I treated the evaluation results as evidence rather than assuming the implementation was reliable.

My engineering decisions: I kept simulated provider results separate from real Groq measurements, retained unsupported-question refusal behaviour, and avoided presenting mock-provider results as actual LLM performance.

My contribution: I worked on the pipeline, retrieval configurations, provider gateway, rate-limit handling, retry behaviour, evaluation experiments, testing, and analysis of the results. I used the assistant to accelerate development and investigate problems, while validating the implementation through tests and recorded experiments.

Reproducibility: `results.csv` combines baseline, non-superseded raw load/retry/fault runs, both real Setting-A quality runs, and the latest A/B/C mock runs. Retrieval metrics come from `results/raw/rb-2.rows.csv`. Missing measurements remain blank. The uploaded source archive was not modified.